TP 5 – Gestion de contacts

Explications du code pour l'oral

Spring Boot · JPA / Hibernate · MySQL · Thymeleaf

Sommaire

- 1. Le fonctionnement général du projet
- 2. Les annotations (les « @ »)
- 3. Les pages Thymeleaf
- 4. Les questions que le prof peut poser

1. Le fonctionnement général du projet

L'idée générale : le modèle MVC

L'application est en **Spring Boot** et suit le principe **MVC** :

- **Modèle** : les classes `Contact` et `Personne`, qui deviennent des tables dans MySQL.
- **Vue** : les pages HTML avec **Thymeleaf**, dans `templates/`.
- **Contrôleur** : les classes `Ctr` et `CtrPersonne`. Elles reçoivent les requêtes du navigateur et choisissent la page à afficher.

Entre le contrôleur et la base, il y a les **repositories** (le package `dao`). Ce sont eux qui lisent et écrivent dans MySQL.

> *À dire à l'oral : « Quand j'ouvre une URL, Spring appelle la bonne méthode du contrôleur. Cette méthode récupère les données avec le repository, les met dans le Model, et renvoie le nom de la page HTML. Thymeleaf affiche ensuite les données dans la page. »*

Organisation des fichiers

Package / dossier	Contenu
`com.example.demo`	`DemoApplication` : la classe qui lance l'application
`com.example.demo.entities`	`Contact`, `Personne` : les tables de la base
`com.example.demo.dao`	`ContactRepository`, `PersonneRepository` : l'accès à la base
`com.example.demo.web`	`Ctr` (contacts), `CtrPersonne` (personnes) : les contrôleurs
`resources/templates`	les pages HTML Thymeleaf
`resources/static/css`	`style.css` : le style de la navbar et des tableaux
`application.properties`	la configuration de la connexion à MySQL

Les entités (package entities)

Personne

```
@Entity
public class Personne {
    @Id @GeneratedValue
    private Long id;
    private String nom;
    private String prenom;
    private String adresse;

    @OneToMany(mappedBy="personne", cascade=CascadeType.REMOVE)
    private List<Contact> contacts;
```

- `@Entity` : Hibernate crée une table `personne` dans MySQL.
- `@Id @GeneratedValue` : l'id est la clé primaire, et elle est générée toute seule (1, 2, 3…).
- `@OneToMany` : **une** personne a **plusieurs** contacts, comme le « 0..* » du diagramme UML.

- `mappedBy="personne"` : c'est la table `contact` qui contient la clé étrangère vers la personne.
- `cascade=REMOVE` : si on supprime une personne, ses contacts sont supprimés aussi. Sinon on aurait des contacts sans propriétaire.

La classe a aussi une méthode `getMoyenne()`. Elle additionne les notes des contacts avec une boucle `for`, puis divise par le nombre de contacts. Si la personne n'a aucun contact, elle renvoie 0.

Contact

```
@Entity
public class Contact {
    @Id
    private String email;
    private int note;

    @ManyToOne
    private Personne personne;
```

- L'**email est la clé primaire**. Le PDF demande qu'un contact soit identifié par un email unique, et une clé primaire est forcément unique.
- `@ManyToOne` : **plusieurs** contacts appartiennent à **une** personne. C'est l'autre côté de la relation, et ça crée une colonne `personne_id` dans la table `contact`.

Les getters et setters servent à Spring et à Thymeleaf. Par exemple, quand on écrit `${contact.email}` dans une page, Thymeleaf appelle `getEmail()`.

Les repositories (package dao)

```
public interface ContactRepository extends JpaRepository<Contact,String>
public interface PersonneRepository extends JpaRepository<Personne,Long>
```

- Ce sont des **interfaces sans code**. Spring écrit lui-même l'implémentation.
- En héritant de `JpaRepository`, on obtient directement `findAll()`, `findById()`, `save()`, `deleteById()` et `existsById()`.
- `<Contact,String>` veut dire : l'entité est `Contact` et le type de sa clé est `String` (l'email).

Il y a aussi des **méthodes dont le nom suffit** : Spring lit le nom et fabrique la requête SQL tout seul.

Méthode	Requête générée
`findByNoteLessThan(10)`	les contacts dont la note est < 10
`findByNote(15)`	les contacts dont la note = 15
`findByContactsIsEmpty()`	les personnes qui n'ont aucun contact

Les contrôleurs (package web)

Ils fonctionnent tous de la même façon :

```
@RequestMapping("/contacts")
public String contacts(Model model) {
    List<Contact> contacts = contactRepository.findAll();  // 1. on récupère les données
    model.addAttribute("contacts", contacts);               // 2. on les envoie à la page
    return "liste";                                          // 3. on affiche liste.html
}
```

- `@Controller` : la classe reçoit les requêtes web.
- `@Autowired` : Spring nous donne automatiquement le repository, on n'a pas besoin de faire `new`.
- `@RequestMapping("/contacts")` : cette méthode est appelée quand on va sur `/contacts`.
- `@RequestParam` : récupère une valeur envoyée par le formulaire, par exemple `name="email"`.
- `Model` : sert à transmettre des données à la page HTML.

Les fonctionnalités, une par une

Ajouter un contact (/ajout puis /save)

- `/ajout` affiche le formulaire. Il envoie aussi la liste des personnes pour remplir la liste déroulante.
- `/save` vérifie d'abord avec `existsById(email)` que l'email n'existe pas déjà. Si c'est le cas, on affiche un message d'erreur.
- Sinon, on récupère la personne choisie, on crée le `Contact` et on fait `save()`.

Modifier un contact (/modifierContact puis /updateContact)

- On récupère le contact avec son email, et le formulaire est **pré-rempli** grâce à `th:value`.
- À l'envoi, on change la note et la personne avec les setters, puis on fait `save()`. Comme le contact existe déjà, `save()` fait une mise à jour et non un ajout.
- L'email est en `readonly` : c'est la clé primaire, on ne doit pas la modifier.

Supprimer un contact (/deleteSave)

On vérifie que le contact existe avec `findById`, puis on fait `deleteById`.

Les personnes (CtrPersonne)

C'est exactement le même principe (liste, ajout, modification, suppression). Pour la modification, l'id est envoyé dans un `<input type="hidden">`, comme ça on sait quelle personne modifier.

Contacts mal notés (/malNotes)

Un simple appel à `findByNoteLessThan(10)`.

Recherche par note (/recherche)

Le paramètre est `required = false`. Si on arrive sur la page sans note, on affiche juste le formulaire. Si une note est donnée, on cherche avec `findByNote(note)`.

Propriétaire d'un contact (/proprietaire)

On cherche le contact par son email, puis `contact.getPersonne()` donne directement la personne grâce au `@ManyToOne`.

Personnes sans contact (/sansContact)

Un appel à `findByContactsIsEmpty()`.

Meilleure moyenne (/meilleureMoyenne)

```
Personne meilleure = null;
for (Personne p : personnes) {
    if (p.getContacts().size() > 0) {
        if (meilleure == null || p.getMoyenne() > meilleure.getMoyenne())
            meilleure = p;
    }
}
```

> *À dire à l'oral : « On parcourt toutes les personnes. On ignore celles qui n'ont pas de contact. À chaque fois qu'une personne a une moyenne plus grande que la meilleure trouvée jusque-là, elle la remplace. C'est un simple calcul de maximum. »*

La configuration (application.properties)

- `spring.datasource.url=...3306/Contacts` : l'adresse de la base MySQL.
- `username=root` et un mot de passe vide : la connexion à MySQL.
- `ddl-auto=create` : Hibernate **crée les tables tout seul** à partir des entités. Attention, il les recrée à chaque démarrage, donc les données sont perdues.

2. Les annotations (les « @ »)

Une annotation, c'est une **étiquette** qu'on colle au-dessus d'une classe, d'un attribut ou d'une méthode. Elle ne fait rien toute seule : elle **donne une information à Spring ou à Hibernate**, qui agissent ensuite à notre place.

> *À dire à l'oral : « Les annotations servent à dire à Spring ce qu'il doit faire, pour éviter d'écrire le code nous-mêmes. »*

Les annotations de la base de données

Elles viennent de **JPA** (`jakarta.persistence`). Elles disent à Hibernate comment transformer une classe Java en table MySQL.

@Entity

```
@Entity
public class Personne {
```

- Hibernate sait que cette classe correspond à **une table** dans la base.
- La classe `Personne` donne la table `personne`, et chaque attribut (`nom`, `prenom`…) devient une **colonne**.
- Sans `@Entity`, la classe reste une classe Java normale, et rien n'est enregistré dans la base.

@Id

```
@Id
private Long id;
```

- Cet attribut est la **clé primaire**, c'est-à-dire l'identifiant unique de chaque ligne.
- Deux lignes ne peuvent pas avoir la même valeur.
- Dans `Contact`, c'est l'**email** qui porte le `@Id`, donc deux contacts ne peuvent pas avoir le même email.

@GeneratedValue

```
@Id @GeneratedValue
private Long id;
```

- La valeur de l'id est **générée automatiquement** (1, 2, 3…). On n'a jamais besoin de la choisir.
- C'est pour ça que le constructeur `new Personne(nom, prenom, adresse)` ne prend pas d'id.
- Dans `Contact`, il n'y a **pas** de `@GeneratedValue`, parce que c'est l'utilisateur qui saisit l'email.

@OneToMany

```
@OneToMany(mappedBy="personne", cascade=CascadeType.REMOVE)
private List<Contact> contacts;
```

- Ça veut dire : **une** (One) personne a **plusieurs** (Many) contacts. C'est pour ça que l'attribut est une `List`.
- `mappedBy="personne"` : « la relation est déjà gérée par l'attribut `personne` dans la classe `Contact` ». Il n'y a donc pas de colonne en plus dans la table `personne`.
- `cascade=CascadeType.REMOVE` : quand on supprime une personne, ses contacts sont supprimés aussi.

@ManyToOne

```
@ManyToOne
private Personne personne;
```

- C'est l'autre côté de la relation : **plusieurs** (Many) contacts appartiennent à **une** (One) personne.
- C'est ce côté-là qui crée la **clé étrangère** : une colonne `personne_id` dans la table `contact`.

> *À dire à l'oral : pour s'en souvenir, on lit l'annotation en partant de la classe où elle est écrite. Dans Personne, @OneToMany se lit « 1 personne → plusieurs contacts ». Dans Contact, @ManyToOne se lit « plusieurs contacts → 1 personne ».*

Les annotations Spring

@SpringBootApplication

```
@SpringBootApplication
public class DemoApplication {
```

- Elle marque la **classe de démarrage** de l'application.
- Au lancement, Spring parcourt tous les packages en dessous de `com.example.demo` pour trouver les contrôleurs, les entités et les repositories. Il configure aussi le reste tout seul : le serveur web, la connexion MySQL et Thymeleaf.

@Controller

```
@Controller
public class Ctr {
```

- Cette classe **reçoit les requêtes du navigateur**.
- Ses méthodes renvoient le **nom d'une page HTML**. Par exemple, `return "liste";` affiche `liste.html`.

@Autowired

```
@Autowired
ContactRepository contactRepository;
```

- Spring **crée l'objet et le met dans la variable** à notre place. C'est ce qu'on appelle l'injection de dépendances.
- Sans `@Autowired`, la variable resterait `null`. Et on ne peut pas faire `new ContactRepository()`, parce que c'est une interface.

@RequestMapping

```
@RequestMapping("/contacts")
public String contacts(Model model) {
```

- Elle **relie une URL à une méthode**.
- Quand on va sur `http://localhost:8080/contacts`, Spring appelle cette méthode.
- Elle accepte le GET (clic sur un lien) comme le POST (envoi de formulaire).

@RequestParam

```
public String save(@RequestParam String email, @RequestParam String note, ...)
```

- Elle **récupère une valeur envoyée par le formulaire ou présente dans l'URL**.
- Le nom de la variable doit correspondre au `name` du champ HTML : `<input name="email">` → `@RequestParam String email`.
- Ça marche aussi pour les URL comme `/modifierPersonne?id=3`, qui donnent `@RequestParam Long id`, avec `id = 3`.
- Avec `@RequestParam(required = false)`, le paramètre est **facultatif**. S'il n'est pas envoyé, il vaut `null`. On s'en sert dans `/recherche` et `/proprietaire` pour pouvoir afficher la page avant que l'utilisateur ait cherché quelque chose.

Les autres annotations

@Override

```
@Override
public String toString() {
```

- Elle indique qu'on **redéfinit une méthode qui existe déjà** dans la classe parente. Ici c'est `toString()`, qui vient de `Object`.
- Si on se trompe dans le nom de la méthode, le compilateur affiche une erreur.

@Test et @SpringBootTest

- `@SpringBootTest` : lance toute l'application pour faire un test.
- `@Test` : marque une méthode comme un test. Ici, le test vérifie simplement que l'application démarre sans erreur.

Récap à retenir

Annotation	En une phrase
`@Entity`	la classe devient une table
`@Id`	la clé primaire
`@GeneratedValue`	l'id est généré automatiquement
`@OneToMany`	1 objet en a plusieurs autres (une `List`)
`@ManyToOne`	plusieurs objets sont liés à 1 seul (clé étrangère)
`@SpringBootApplication`	la classe qui lance l'application
`@Controller`	la classe qui reçoit les requêtes web
`@Autowired`	Spring crée l'objet à notre place
`@RequestMapping`	relie une URL à une méthode
`@RequestParam`	récupère une valeur du formulaire ou de l'URL
`@Override`	on redéfinit une méthode héritée

3. Les pages Thymeleaf

C'est quoi Thymeleaf ?

Thymeleaf est un **moteur de templates**. On écrit une page HTML normale et on y ajoute des attributs spéciaux qui commencent par `th:`. Avant d'envoyer la page au navigateur, Thymeleaf **remplace ces attributs par les vraies données** : la liste des contacts, le nom d'une personne, etc.

> *À dire à l'oral : « Thymeleaf permet de mettre des données Java dans une page HTML. Le contrôleur envoie les données avec le Model, et la page les affiche avec les attributs th:. »*

Les pages sont dans `src/main/resources/templates/`. Quand le contrôleur fait `return "liste";`, Spring affiche `templates/liste.html`.

Le navigateur ne voit jamais les `th:`. Il reçoit du HTML normal, déjà rempli.

La ligne du haut

```
<html xmlns:th="http://www.thymeleaf.org">
```

Elle déclare les attributs `th:`. C'est ce qui permet d'écrire `th:text`, `th:each`, etc. dans la page.

Les 3 symboles à connaître

Symbole	Sert à	Exemple
`${...}`	lire une **donnée** envoyée par le contrôleur	`${contact.email}`
`@{...}`	construire une **URL**	`@{/contacts}`
`~{...}`	insérer un **morceau d'une autre page** (un fragment)	`~{fragments/navbar :: navbar}`

${...}, les données

Dans le contrôleur :

```
model.addAttribute("contacts", contacts);
```

Dans la page, `${contacts}` donne cette liste. Le nom doit être **le même des deux côtés**.

Quand on écrit `${contact.email}`, Thymeleaf appelle `contact.getEmail()`. C'est pour ça qu'il faut des getters dans les entités.

On peut aussi enchaîner : `${contact.personne.nom}` revient à `contact.getPersonne().getNom()`.

@{...}, les URL

```
<a th:href="@{/contacts}">Liste des contacts</a>
```

Avec des paramètres :

```
<a th:href="@{/modifierContact(email=${contact.email})}">Modifier</a>
```

Thymeleaf génère `/modifierContact?email=a@a.fr`. C'est la valeur que le contrôleur récupère avec `@RequestParam String email`.

Les attributs th: utilisés dans le projet

th:text, afficher une valeur

```
<td th:text="${contact.email}" />
```

Le contenu de la case est remplacé par l'email, ce qui donne `<td>a@a.fr</td>`. On peut aussi coller du texte :

```
<td th:text="${contact.personne.prenom + ' ' + contact.personne.nom}" />
```

Ici on obtient « Jean Dupont ».

th:each, faire une boucle

```
<tr th:each="contact: ${contacts}">
    <td th:text="${contact.email}" />
    <td th:text="${contact.note}" />
</tr>
```

- Ça marche comme un `for (Contact contact : contacts)` en Java.
- La ligne `<tr>` est **répétée une fois par contact**.
- Dans la boucle, la variable `contact` est le contact en cours.

On l'utilise dans tous les tableaux (liste des contacts, personnes, mal notés…) et dans les listes déroulantes :

```
<select name="personneId">
    <option th:each="p : ${personnes}" th:value="${p.id}"
            th:text="${p.prenom + ' ' + p.nom}"></option>
</select>
```

On obtient une `<option>` par personne. On **voit** le nom, mais c'est l'**id** qui est envoyé au contrôleur.

th:if, afficher seulement si la condition est vraie

```
<div th:if="${contacts != null}">
    <table>...</table>
</div>
```

Dans `recherche.html`, le tableau n'apparaît **qu'après une recherche**. Avant, `contacts` vaut `null`, parce que le contrôleur ne l'a pas envoyé. Même principe dans `proprietaire.html` et `meilleure.html` avec `${personne != null}`.

th:action, l'adresse du formulaire

```
<form th:action="@{/save}" method="post">
    <input type="text" name="email">
    <input type="text" name="note">
    <input type="submit" value="Envoyer">
</form>
```

- Quand on clique sur « Envoyer », les données partent vers `/save`.
- Le `name` de chaque champ correspond au `@RequestParam` du contrôleur. Par exemple, `name="email"` donne `@RequestParam String email`.

th:value, pré-remplir un champ

```
<input type="text" name="note" th:value="${contact.note}">
```

On s'en sert dans les pages **de modification**, pour que le champ contienne déjà l'ancienne valeur. Dans `modifPersonne.html`, l'id est dans un champ caché :

```
<input type="hidden" name="id" th:value="${personne.id}">
```

L'utilisateur ne le voit pas, mais il est envoyé. Le contrôleur sait ainsi **quelle personne modifier**.

th:selected, choisir l'option par défaut

```
<option th:each="p : ${personnes}" ...
        th:selected="${p.id == contact.personne.id}"></option>
```

Dans `modifContact.html`, la liste déroulante affiche directement **la personne actuelle du contact**.

th:fragment et th:replace, la navbar

Dans `fragments/navbar.html`, on donne un nom au morceau de page :

```
<div class="navbar" th:fragment="navbar">
    ...les liens...
</div>
```

Dans chaque page, on l'insère :

```
<div th:replace="~{fragments/navbar :: navbar}"></div>
```

- `fragments/navbar` est le fichier, et `:: navbar` est le nom du fragment.
- Ce `<div>` est **remplacé** par la navbar.
- L'intérêt : la navbar est écrite **une seule fois**. Pour ajouter un lien, on ne modifie qu'un fichier.

Les pages une par une

Page	URL	Ce qu'elle fait
`index.html`	`/`	page d'accueil
`liste.html`	`/contacts`	tableau de tous les contacts + liens Modifier / Supprimer
`add.html`	`/ajout`	formulaire d'ajout de contact + liste déroulante des personnes
`modifContact.html`	`/modifierContact`	formulaire pré-rempli (`th:value`, `th:selected`)
`delete.html`	`/delete`	formulaire pour supprimer un contact avec son email
`personnes.html`	`/personnes`	tableau des personnes + nombre de contacts

Page	URL	Ce qu'elle fait
`addPersonne.html`	`/ajoutPersonne`	formulaire d'ajout de personne
`modifPersonne.html`	`/modifierPersonne`	formulaire pré-rempli + id caché
`malnotes.html`	`/malNotes`	contacts avec une note < 10
`recherche.html`	`/recherche`	formulaire + résultat de la recherche par note
`proprietaire.html`	`/proprietaire`	trouve la personne d'un contact
`sanscontact.html`	`/sansContact`	personnes qui n'ont aucun contact
`meilleure.html`	`/meilleureMoyenne`	personne avec la meilleure moyenne

Dans `meilleure.html`, `${personne.moyenne}` appelle la méthode `getMoyenne()` de la classe `Personne`. Thymeleaf la traite comme un attribut grâce au mot « get ».

Un exemple complet du début à la fin

Voici ce qui se passe quand on **ajoute un contact** :

- **1.** On clique sur « Ajouter un contact » dans la navbar, ce qui ouvre `/ajout`.
- **2.** Le contrôleur envoie la liste des personnes avec `model.addAttribute("personnes", ...)` et renvoie `"add"`.
- **3.** Dans `add.html`, `th:each` remplit la liste déroulante avec les personnes.
- **4.** On remplit le formulaire et on clique sur « Envoyer ». Les données partent vers `/save`, à cause de `th:action="@{/save}"`.
- **5.** Le contrôleur récupère `email`, `note` et `personneId` avec `@RequestParam`, puis enregistre le contact.
- **6.** Il renvoie la page `liste.html`, où `th:each` affiche tous les contacts, y compris le nouveau.

4. Les questions que le prof peut poser

Sur la base de données

Pourquoi l'email en clé primaire ?

Parce que le PDF dit qu'un contact est identifié par un email unique. Une clé primaire est unique par définition.

Que se passe-t-il si on supprime une personne ?

Ses contacts sont supprimés aussi, grâce à `cascade = REMOVE`.

Où est le SQL ?

Il n'y en a pas à écrire : Spring Data JPA le génère à partir du nom des méthodes, et Hibernate crée les tables.

Quelle différence entre @OneToMany et @ManyToOne ?

Ce sont les deux côtés de la même relation : une personne a plusieurs contacts, et un contact a une seule personne. C'est le côté `@ManyToOne` qui contient la clé étrangère.

Comment tu évites deux contacts avec le même email ?

Dans `/save`, `existsById(email)` vérifie avant d'enregistrer. Si l'email existe déjà, on affiche un message d'erreur.

Sur Spring

À quoi sert @Autowired ?

Spring crée le repository et le met dans la variable à notre place. On ne peut pas faire `new` sur une interface.

Comment Spring sait quelle méthode appeler ?

Grâce à `@RequestMapping("/url")` au-dessus de chaque méthode du contrôleur.

Comment tu calcules la meilleure moyenne ?

Une boucle `for` sur toutes les personnes qui ont au moins un contact. On garde celle qui a la plus grande moyenne, calculée par `getMoyenne()` dans la classe `Personne`.

Sur Thymeleaf

Comment la page reçoit les données ?

Le contrôleur les met dans le `Model` avec `addAttribute`, et la page les lit avec `${...}`.

Pourquoi utiliser @{...} et pas écrire l'URL directement ?

Thymeleaf construit l'URL correctement, y compris les paramètres comme `?email=...`.

Comment tu évites de répéter la navbar ?

Elle est dans un fragment (`th:fragment`) que chaque page insère avec `th:replace`.

Comment le contrôleur sait quel champ est quoi ?

Grâce à l'attribut `name` des `<input>`, qui doit être le même que le nom du `@RequestParam`.